

Classical Forecasting Techniques

Dr. Sankalp Purushottam Naik

Classical forecasting techniques are a set of methods used to predict future values based on historical data. These techniques are widely used in various fields, including finance, economics, and supply chain management. In this section, we will explore some of the most commonly used classical forecasting techniques, including naive methods, moving average methods, exponential smoothing methods, Holt method, and Holt-Winters methods. We will also discuss their advantages and limitations, as well as how to implement them in practice.

Data Generating Process (DGP)

A data generating process (DGP) is a statistical model that describes how data is generated. It is a theoretical construct that helps us understand the underlying patterns and relationships in the data. In practice, we often do not know the true DGP, and we must rely on assumptions and approximations to make predictions.

Examples of DGPs include linear regression models, time series models, and stochastic processes. Understanding the DGP is crucial for selecting appropriate forecasting techniques and for interpreting the results of forecasts.

True DGP vs. Models

True DGP refers to the actual process that generates the data, while models are simplified representations of the DGP that we use for forecasting. Models may not capture all the complexities of the true DGP, but they can still provide useful insights and predictions. It is important to evaluate the performance of models against the true DGP to ensure their accuracy and reliability.

Illustration 1: DGP and Models - Study Hours and Exam Scores

Suppose we observe the following data for 10 students. Each student reports the number of hours they studied before an examination, and we observe their actual examination score.

Student	Study Hours	Actual Score
1	1.5	43
2	2.0	47
3	3.5	56
4	4.0	59
5	5.5	68
6	6.0	70

Student	Study Hours	Actual Score
7	7.5	78
8	8.0	81
9	9.5	87
10	10.0	89

At this stage, **we make no assumptions about how the data were generated.**

We simply observe the relationship between study hours and actual scores.

Data Display and Scatter Plot

Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt

# Create the dataset

df = pd.DataFrame({
    "Student": range(1, 11),
    "Study_Hours": [1.5, 2.0, 3.5, 4.0, 5.5,
                    6.0, 7.5, 8.0, 9.5, 10.0],
    "Actual_Score": [43, 47, 56, 59, 68,
                    70, 78, 81, 87, 89]
})

# Display the data

display(df)

# Create scatter plot

plt.figure(figsize=(9, 5.5))

plt.scatter(
    df["Study_Hours"],
    df["Actual_Score"],
    s=80
)

plt.xlabel("Study Hours", fontsize=12)
plt.ylabel("Actual Score", fontsize=12)
plt.title(
    "Relationship Between Study Hours and Exam Scores",
    fontsize=14,
    pad=15
)

plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

Program 1: Data Display and Scatter Plot

(a)

	Student	Study_Hours	Actual_Score	
0	1	1.5	43	4
1	2	2.0	47	
2	3	3.5	56	
3	4	4.0	59	

What do we see?

The scatter plot suggests a **positive relationship** between study hours and examination scores.

Students who study more tend to obtain higher scores.

However, the relationship is **not perfectly deterministic**.

For example, students who study a similar number of hours do not necessarily obtain exactly the same score.

This suggests that something systematic is happening, but there is also some randomness in the data.

Signal and Noise

We can think of the observed score as consisting of two components:

$$\boxed{\text{Observed Data} = \text{Signal} + \text{Noise}}$$

Signal

The **signal** is the systematic pattern in the data.

In our example, the signal is the tendency for examination scores to increase as study hours increase.

Noise

The **noise** is the random variation around the systematic pattern.

Even if two students study for approximately the same amount of time, their scores may differ because of:

- Prior knowledge
- Individual ability
- Concentration
- Exam difficulty
- Luck
- Other factors

Therefore, we do not expect every observation to lie exactly on the underlying relationship.

Capturing the Signal with Linear Regression

One simple way to capture the systematic component of the relationship is **linear regression**.

We specify:

$$Score_i = \beta_0 + \beta_1 StudyHours_i + \varepsilon_i$$

where:

- β_0 represents the intercept
- β_1 represents the effect of study hours

- ε_i represents the noise

The regression model attempts to identify the **signal** while treating the remaining variation as noise.

In estimated form:

$$\widehat{Score}_i = \hat{\beta}_0 + \hat{\beta}_1 StudyHours_i$$

Estimate the Model

💡 Coding Session

Program 2: Linear Regression: Capturing the Signal

```
import statsmodels.api as sm

# Define dependent and independent variables

X = df["Study_Hours"]
y = df["Actual_Score"]

# Add intercept

X = sm.add_constant(X)

# Estimate the regression model

model = sm.OLS(y, X).fit()

# Display regression output

print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Actual_Score   R-squared:                0.994
Model:                  OLS           Adj. R-squared:           0.994
Method:                 Least Squares   F-statistic:              1423.
Date:                  Tue, 29 Sep 2026   Prob (F-statistic):       2.68e-10
Time:                  01:14:15         Log-Likelihood:           -15.587
No. Observations:      10              AIC:                     35.17
Df Residuals:          8               BIC:                     35.78
Df Model:              1
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	36.7411	0.918	40.011	0.000	34.624	38.859
Study_Hours	5.4016	0.143	37.723	0.000	5.071	5.732

```
=====
Omnibus:                1.779   Durbin-Watson:              0.672
Prob(Omnibus):          0.411   Jarque-Bera (JB):          0.996
Skew:                   -0.447   Prob(JB):                  0.608
Kurtosis:               1.739   Cond. No.                  14.8
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Regression Line and Equation

We can now plot the observed data together with the estimated regression line.

💡 Coding Session

```
import numpy as np
import matplotlib.pyplot as plt

# Generate fitted values

x = np.linspace(
    df["Study_Hours"].min(),
    df["Study_Hours"].max(),
    100
)

b0 = model.params["const"]
b1 = model.params["Study_Hours"]

y_hat = b0 + b1 * x

# Create the plot

plt.figure(figsize=(9, 5.5))

plt.scatter(
    df["Study_Hours"],
    df["Actual_Score"],
    s=80,
    label="Observed data"
)

plt.plot(
    x,
    y_hat,
    linewidth=2.5,
    label="Estimated signal"
)

# Regression equation

equation = f"Score = {b0:.2f} + {b1:.2f} × Study Hours"

plt.text(
    0.05,
    0.93,
```

Program 3
equation,
transform=plt.gca().transAxes,
fontsize=12

10
Observed Scores and the Estimated Signal



Interpreting the Model

The estimated regression equation provides a simple representation of the systematic relationship between study hours and examination scores.

Intercept

The intercept represents the model's predicted score when study hours are zero.

Slope

The slope represents the expected change in examination score associated with one additional hour of study.

A positive slope indicates that **higher study hours are associated with higher examination scores**.

Residuals

The difference between the actual score and the score predicted by the regression model is the **residual**:

$$e_i = Score_i - \widehat{Score}_i$$

These residuals represent the portion of the observed data that is **not explained by the systematic relationship captured by the model**.

From Model to DGP

We can now return to our original question:

What process generated the observed data?

A simple representation is:

$$Score_i = \beta_0 + \underbrace{\beta_1 StudyHours_i}_{\text{Signal}} + \underbrace{\varepsilon_i}_{\text{Noise}}$$

The regression model gives us an **estimate of the systematic component**, while the residuals provide an estimate of the unexplained component.

The Data Generating Process (DGP) is the underlying process that generates the observed data.

In this example, we conceptualize the DGP as a systematic relationship between study hours and examination scores, together with a random component.

The Big Picture

The important distinction is:

$$\boxed{\text{DGP} \rightarrow \text{Signal} + \text{Noise} \rightarrow \text{Observed Data}}$$

We observe the **data**.

We try to infer the underlying **signal**.

What remains unexplained is treated as **noise**.

This idea is fundamental to forecasting.

Forecasting is essentially an attempt to identify the systematic component of a data-generating process and use it to predict future observations.

In time-series forecasting, the signal may take different forms:

- **Level**
- **Trend**
- **Seasonality**
- **Cycles**

The challenge is to distinguish these systematic patterns from random noise.

Naïve forecasting Techniques

Naïve forecasting techniques are simple methods that use historical data to make predictions about future values. These techniques are often used as a baseline for comparison with more complex forecasting methods. In this section, we will explore some of the most commonly used naïve forecasting techniques, including the naïve method, seasonal naïve method, and drift method. We will also discuss their advantages and limitations, as well as how to implement them in practice.

Naïve Method

Naive method is a simple forecasting technique that assumes that the future value of a time series will be equal to the most recent observed value. This method is often used as a benchmark for evaluating the performance of more complex forecasting methods. It is particularly useful for time series data that exhibit a random walk behavior, where the best prediction for the next value is simply the last observed value.

Example: Naïve Forecasting

Suppose we observe the following monthly closing prices of a stock:

Month	Month-End Stock Price (₹)
January	1,842.50
February	1,875.25
March	1,821.75

Month	Month-End Stock Price (₹)
April	1,906.50
May	1,958.25
June	1,932.75
July	2,014.50
August	2,067.25
September	2,041.50
October	?

Question

Using the **Naïve Forecasting Method**, what would you predict the October month-end stock price to be?

Answer

Visualizing the Data

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt

# Create the dataset
df = pd.DataFrame(
    {
        "Month": [
            "January",
            "February",
            "March",
            "April",
            "May",
            "June",
            "July",
            "August",
            "September",
        ],
        "Stock_Price": [
            1842.50,
            1875.25,
            1821.75,
            1906.50,
            1958.25,
            1932.75,
            2014.50,
            2067.25,
            2041.50,
        ],
    },
)

# Plot monthly stock prices
plt.figure(figsize=(10, 5))

plt.plot(df["Month"], df["Stock_Price"], marker="o")

plt.xlabel("Month")
plt.ylabel("Month-End Stock Price (₹)")
plt.title("Monthly Stock Price")

plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
```

Program 4



The naïve method assumes that the forecast for the next period will be equal to the **actual value observed in the current period**.

$$F_t = Y_{t-1}$$

Therefore:

$$F_{\text{October}} = Y_{\text{September}}$$

$$F_{\text{October}} = 2,041.50$$

The October forecast is ₹2,041.50.

Seasonal Naïve Method

The seasonal naïve method is an extension of the naïve method that takes into account the seasonality of the data. It assumes that the forecast for a given period will be equal to the actual value observed in the same period of the previous season. This method is particularly useful for time series data that exhibit strong seasonal patterns, such as monthly sales data or quarterly revenue data.

Example: Seasonal Naïve Forecasting

Suppose the quarterly sales of a company are as follows:

Year	Quarter	Sales (₹ lakh)
2024	Q1	100
2024	Q2	120
2024	Q3	105
2024	Q4	145
2025	Q1	98
2025	Q2	122
2025	Q3	103
2025	Q4	148
2026	Q1	?
2026	Q2	?
2026	Q3	?
2026	Q4	?

Question

Using the **Seasonal Naïve Forecasting Method**, what would you forecast for each quarter of 2026?

Answer

Visualizing the Data

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt

# Create the dataset
df = pd.DataFrame({
    "Year": [2024]*4 + [2025]*4,
    "Quarter": ["Q1", "Q2", "Q3", "Q4"] * 2,
    "Sales": [100, 120, 105, 145, 98, 122, 103, 148]
})

# Create a sequential time-period variable
df["Period"] = range(1, len(df) + 1)

# Plot quarterly sales
plt.figure(figsize=(10, 5))

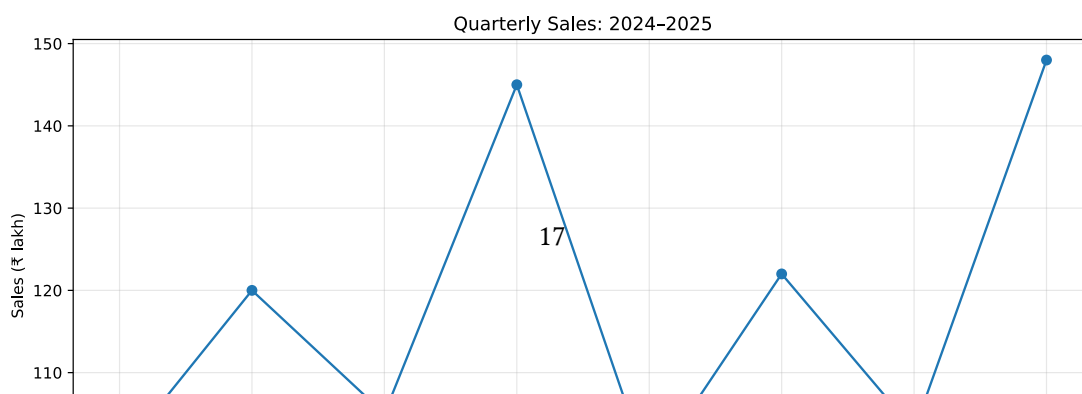
plt.plot(
    df["Period"],
    df["Sales"],
    marker="o"
)

plt.xticks(
    df["Period"],
    [f"{year} {quarter}" for year, quarter in zip(df["Year"], df["Quarter"])]
)

plt.xlabel("Quarter")
plt.ylabel("Sales (₹ lakh)")
plt.title("Quarterly Sales: 2024–2025")

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Program 5



For quarterly data, the seasonal cycle is **4 quarters**:

$$s = 4$$

Therefore, the seasonal naïve forecast is:

$$F_t = Y_{t-4}$$

So:

Quarter	2026 Forecast (₹ lakh)	Based On
Q1	98	2025 Q1
Q2	122	2025 Q2
Q3	103	2025 Q3
Q4	148	2025 Q4

Key idea: Instead of using the **immediately preceding observation**, the seasonal naïve method uses the observation from the **same season in the previous cycle**.

Smoothing Forecasting Techniques

Smoothing forecasting techniques are a set of methods that use weighted averages of past observations to make predictions about future values. These techniques are particularly useful for time series data that exhibit trends or seasonality. In this section, we will explore some of the most commonly used smoothing forecasting techniques, including moving average methods, exponential smoothing methods, Holt method, and Holt-Winters methods. We will also discuss their advantages and limitations, as well as how to implement them in practice.

Moving Average Methods

Moving average methods are a simple smoothing technique that calculates the average of a fixed number of past observations to make predictions about future values. The moving average can be calculated using different window sizes, which determines how many past observations are included in the average. This method is particularly useful for time series data that exhibit short-term fluctuations or noise, as it helps to smooth out the data and reveal underlying trends.

Short vs Long Moving Average Window

	 Short Window	 Long Window
 Observations	Fewer observations	More observations
 Responsiveness	High	Low
 Sensitivity to noise	High	Low
 Smoothing	Less smoothing	More smoothing
 Lag	Lower	Higher

Short Window

Long Window

Best when

Recent changes matter

Underlying pattern is stable

The Trade-off

Short Window → More Responsive, Less Smooth

Long Window → More Smooth, Less Responsive

 **Key Insight:** A longer window removes more **noise**, but may also make the forecast slow to react to genuine changes in the **signal**.

Example: Simple Moving Average (SMA) Analysis

In time-series analysis, moving averages help smooth short-term fluctuations and highlight longer-term movements in a series. A shorter moving-average window responds more quickly to recent changes, while a longer window produces a smoother representation of the underlying trend.

Question

Using the quarterly automobile sales dataset (`automobile_sales.csv`), inspect the data structure, display the first few observations, and construct two visualizations:

1. A raw line plot of quarterly automobile sales.
2. A plot showing the original sales data alongside 3-period and 10-period simple moving averages.

Answer

Visualizing the Data

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
file_path = "/home/sankalpnaique/PyWorks/BGF/data/automobile_sales.csv"
df = pd.read_csv(file_path)

# Inspect the dataset
print("First 5 rows of the dataset:")
display(df.head())

print("\nData types:")
display(df.dtypes)

print("\nNumber of observations and variables:")
print(df.shape)

# Calculate 3-period and 10-period moving averages
df["MA_3"] = df["sales"].rolling(window=3).mean()
df["MA_10"] = df["sales"].rolling(window=10).mean()

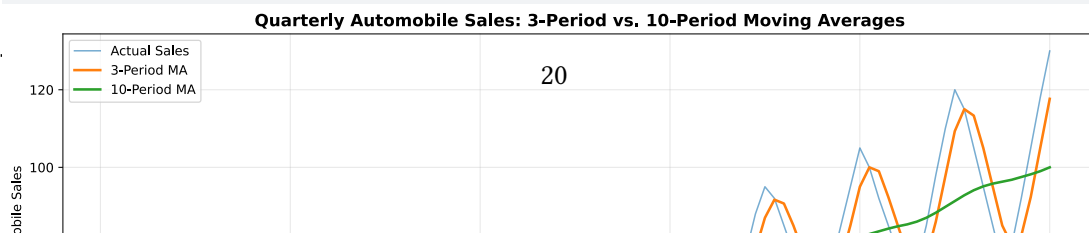
# -----
# Figure 1: Raw quarterly sales
# -----

plt.figure(figsize=(12, 5))

plt.plot(
    df["qtr"],
    df["sales"],
    linewidth=1.5,
    label="Actual Sales"
)

plt.title(
    "Quarterly Automobile Sales",
    fontsize=13,
    fontweight="bold"
)

plt.xlabel("Quarter")
```



The k -period simple moving average at period t is calculated as:

$$\text{MA}_t = \frac{1}{k} \sum_{i=0}^{k-1} Y_{t-i}$$

Key Insight

3-Period Moving Average ($k = 3$): The shorter window responds relatively quickly to changes in automobile sales and therefore retains more short-term movement.

10-Period Moving Average ($k = 10$): The longer window produces greater smoothing and is less sensitive to short-term fluctuations, making it more useful for identifying broader movements in the series.

Weighted Moving Average (WMA)

Weighted Moving Average (WMA) is a variation of the simple moving average that assigns different weights to past observations, giving more importance to recent data points. This method is particularly useful when recent observations are believed to be more relevant for forecasting future values. The weights can be assigned based on various criteria, such as linear or exponential decay.

The core intuition behind the Weighted Moving Average (WMA) comes down to a simple reality of most real-world data: what happened yesterday is usually a better predictor of tomorrow than what happened last month.

While a Simple Moving Average (SMA) treats every period equally—giving an observation from 10 quarters ago the exact same voting power as the most recent quarter—a WMA shifts the balance of power.

The Intuitive Analogy: Performance Reviews

Imagine evaluating a team member's performance:

Simple Moving Average: You take the average of their performance over the last 10 years. An error made in Year 1 penalizes them just as heavily as an error made last week.

Weighted Moving Average: You evaluate the full 10-year period, but you give 80% of the weight to their recent 2 years of work and 20% to the older years. This reflects their current skill level much more accurately.

Why WMA Beats SMA for Recent Shifts

 **Reduces Lag:** Because SMAs treat all points equally, they are slow to react when a trend changes. WMAs “hug” recent structural shifts much faster.

 **Controlled Memory:** By choosing how fast the weights decay (e.g., linear vs. exponential), you control how quickly the model “forgets” distant history.

Mathematical Formulation of WMA

For a 3-period moving average:

$$F_t = \frac{Y_{t-1} + Y_{t-2} + Y_{t-3}}{3}$$

Each observation receives the same weight:

$$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$$

In a Weighted Moving Average, we deliberately give different weights:

$$F_t = w_1 Y_{t-1} + w_2 Y_{t-2} + w_3 Y_{t-3}$$

where:

$$w_1 + w_2 + w_3 = 1$$

For example, we might give the most recent observation the greatest weight:

$$\boxed{F_t = 0.5Y_{t-1} + 0.3Y_{t-2} + 0.2Y_{t-3}}$$

So the intuition is simply:

Moving Average: Every observation matters equally.

Weighted Moving Average: Recent observations can matter more.

Example: Simple vs Weighted Moving Average

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
file_path = "/home/sankalpnaique/PyWorks/BGF/data/automobile_sales.csv"
df = pd.read_csv(file_path)

# 3-period Simple Moving Average
df["SMA_3"] = df["sales"].rolling(window=3).mean()

# 3-period Weighted Moving Average
# Weights: 10% oldest, 30% middle, 60% most recent
weights = [0.10, 0.30, 0.60]

df["WMA_3"] = df["sales"].rolling(window=3).apply(lambda x: sum(x * weights),
raw=True)

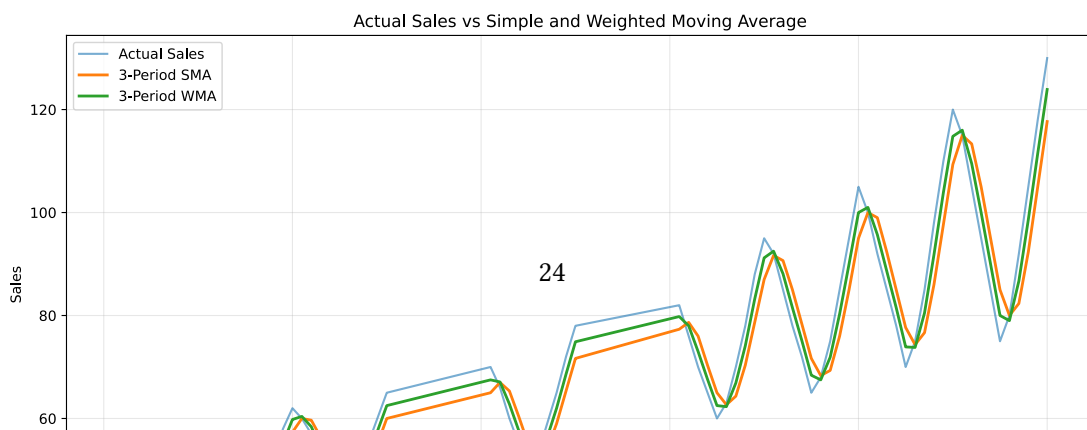
# Plot actual sales, SMA and WMA
plt.figure(figsize=(11, 6))

plt.plot(df["qtr"], df["sales"], label="Actual Sales", alpha=0.6)
plt.plot(df["qtr"], df["SMA_3"], label="3-Period SMA", linewidth=2)
plt.plot(df["qtr"], df["WMA_3"], label="3-Period WMA", linewidth=2)

plt.xlabel("Quarter")
plt.ylabel("Sales")
plt.title("Actual Sales vs Simple and Weighted Moving Average")
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Program 6



Trend & Seasonality Forecasting Techniques

Trend and seasonality forecasting techniques are methods that account for long-term trends and seasonal patterns in time series data. These techniques are particularly useful for data that exhibit systematic changes over time, such as sales data that may increase during certain seasons or years. In this section, we will explore some of the most commonly used trend and seasonality forecasting techniques, including Holt's linear trend method and Holt-Winters seasonal method. We will also discuss their advantages and limitations, as well as how to implement them in practice.

Holt's Exponential Smoothing Model (Double Exponential Smoothing)

Holt's model extends **Simple Exponential Smoothing** by explicitly incorporating a **trend component**.

While Simple Exponential Smoothing is designed for data with a relatively stable level, Holt's model is useful when the series shows a systematic upward or downward movement over time.

The model therefore estimates two components:

-  **Level:** The current underlying value of the series
-  **Trend:** The direction and rate of change over time

The intuition is simple:

Simple Exponential Smoothing: What is the current level? **Holt:** What is the current level + where is it heading?

Mathematical Formulation of Holt's Model

The level and trend are updated recursively:

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

The forecast is:

$$F_{t+h} = L_t + hT_t$$

where:

- α = smoothing parameter for the level
- β = smoothing parameter for the trend
- h = number of periods ahead

Example: Holt's Model

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Load the dataset
file_path = "/home/sankalpnaique/PyWorks/BGF/data/automobile_sales.csv"

df = pd.read_csv(file_path)

# Fit Holt's model with an additive trend
model = ExponentialSmoothing(
    df["sales"],
    trend="add"
).fit()

# Generate fitted values
df["Holt"] = model.fittedvalues

# Plot actual sales and Holt's fitted values
plt.figure(figsize=(11, 6))

plt.plot(
    df["qtr"],
    df["sales"],
    label="Actual Sales",
    alpha=0.6
)

plt.plot(
    df["qtr"],
    df["Holt"],
    label="Holt's Model",
    linewidth=2
)

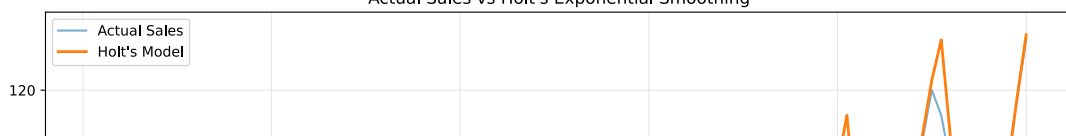
plt.xlabel("Quarter")
plt.ylabel("Sales")
plt.title("Actual Sales vs Holt's Exponential Smoothing")

plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
```

Program 7

plt.tight_layout()

27
Actual Sales vs Holt's Exponential Smoothing



Holt-Winters Model

Holt-Winters extends Holt's model by adding a **seasonal component**.

It is therefore useful when the time series contains both:

-  **Trend**
-  **Seasonality**

For quarterly data, one complete seasonal cycle contains **4 observations**, so:

$$s = 4$$

The intuition is:

Holt: Level + Trend

Holt-Winters: Level + Trend + Seasonality

Example: Holt-Winters Model

💡 Coding Session

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Load the dataset
file_path = "/home/sankalpnaique/PyWorks/BGF/data/automobile_sales.csv"

df = pd.read_csv(file_path)

# Fit Holt-Winters model
# Quarterly data → seasonal cycle of 4 periods
model = ExponentialSmoothing(
    df["sales"], trend="add", seasonal="add", seasonal_periods=4
).fit()

# Generate fitted values
df["Holt_Winters"] = model.fittedvalues

# Plot actual sales and Holt-Winters fitted values
plt.figure(figsize=(11, 6))

plt.plot(df["qtr"], df["sales"], label="Actual Sales", alpha=0.6)

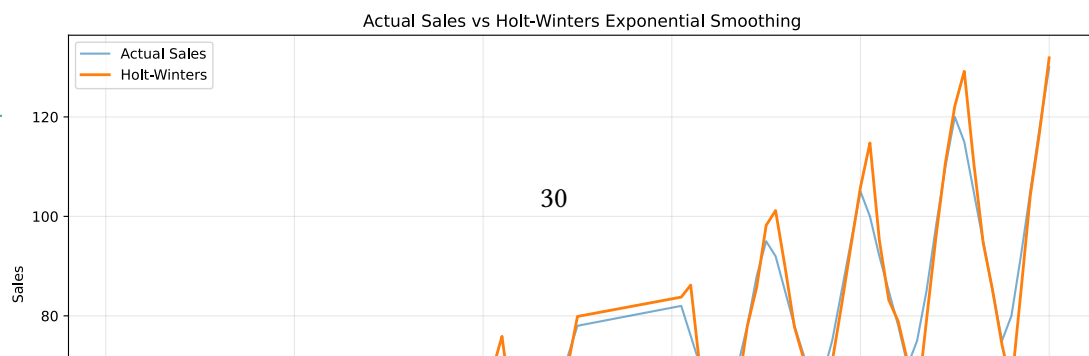
plt.plot(df["qtr"], df["Holt_Winters"], label="Holt-Winters", linewidth=2)

plt.xlabel("Quarter")
plt.ylabel("Sales")
plt.title("Actual Sales vs Holt-Winters Exponential Smoothing")

plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Program 8



The Progression

Model	Components Captured
Simple Exponential Smoothing	 Level
Holt	 Level +  Trend
Holt-Winters	 Level +  Trend +  Seasonality

The key idea: Each model adds another component of the underlying time-series pattern.